# AI Job Application Platform

## Product Requirements Document, HLD and LLD

Version 0.1 | 27 October 2026 | Status: Architecture proposal

## 1. Executive summary

This repository will evolve into a verification-driven job application platform. It will maintain a candidate profile in Markdown, discover relevant LinkedIn posts and job links, reject suspicious or low-confidence postings, tailor a CV and cover letter from evidence, prepare recruiter outreach, and track every decision in an auditable application manifest.

The system is not an autonomous spam bot. It is a human-in-the-loop assistant. Reading, scoring, drafting, document compilation, deterministic verification, and tracking may be automated. Sending an email, publishing a comment, submitting an application, or changing a user profile requires an explicit approval token for that exact action.

## 2. Goals and non-goals

### Goals

- One source of truth for candidate facts, preferences, constraints, writing style, and evidence.
- Configurable discovery from LinkedIn posts and job links using the existing Apify read layer.
- Job relevance ranking using explicit requirements, evidence coverage, freshness, and trust signals.
- Fraud and low-quality detection: duplicate posters, implausible recruiter behavior, missing company identity, suspicious redirects, injection-shaped content, and rate anomalies.
- Tailored CV and cover letter generation with no invented experience.
- PDF compilation, text extraction, visual rendering, hyperlink/font/page checks, and independent review.
- Gmail-assisted status detection with least-privilege, read-only defaults.
- Application manifests, immutable audit events, retries, idempotency, and resumable workflows.

### Non-goals for the first release

- Circumventing LinkedIn controls, CAPTCHAs, rate limits, or terms of service.
- Sending bulk unsolicited messages or comments.
- Automatically completing arbitrary third-party forms without a visible review step.
- Making hiring, salary, immigration, or legal claims on behalf of the candidate.
- Storing secrets in the repository or generated public artifacts.

## 3. Personas and success metrics

The primary persona is a candidate who wants fewer, better applications. Secondary personas are a reviewer who audits generated materials and an operator who maintains connectors and policies.

Success is measured by verified application quality, not volume: percentage of applications with complete evidence maps, zero fabricated claims, PDF pass rate, recruiter-response rate, false-positive fraud rate, time from discovery to approval, and percentage of external actions with an approval record.

## 4. Product workflow

1. Setup imports a CV, LinkedIn export, documents, preferences, target roles, locations, salary range, work authorization, and outreach consent.

2. Discovery fetches public post data through Apify and normalizes text, author, company, timestamp, links, contact emails, and job identifiers.

3. Trust screening strips hidden text, treats fetched content as untrusted data, checks links and identity signals, and assigns a confidence score.

4. Relevance analysis extracts requirements and maps each requirement to candidate evidence: strong, partial, gap, or not applicable.

5. Ranking produces a shortlist with reasons, risks, and the next recommended action.

6. Drafting creates a tailored CV, cover letter, recruiter email, and post comment draft.

7. Independent review checks factual accuracy, requirement coverage, tone, company relevance, and consistency.

8. Verification compiles and renders PDFs, extracts text, checks page count and links, and blocks critical defects.

9. Approval presents a human-readable action bundle. The user approves each external action or rejects it.

10. Execution uses the correct connector and records request, response, confirmation, and idempotency key.

11. Outcome tracking reads approved Gmail signals, records interview/rejection/response states, and archives the final materials.

## 5. Functional requirements

### Candidate profile

The profile is Markdown with stable section identifiers and an optional structured companion JSON. It contains identity/contact data, work authorization, location, role targets, employment history, education, skills with proficiency and evidence, projects, metrics, portfolio links, salary constraints, preferences, availability, behavioral profile, writing style, consent settings, and prohibited claims. Each fact has source, confidence, last verified date, and optional expiry.

### Discovery and controls

Users can configure keywords, title synonyms, companies, locations, remote policy, seniority, recency, salary, industries, poster types, minimum trust score, maximum posts per author per month, and excluded terms. The

default author-frequency rule flags more than two job-like posts from the same person in a 30-day window for review; it is a signal, not proof of fraud.

### Outreach and application

The system detects public recruiter emails, but never sends automatically unless the user has enabled a named policy. It generates a short personalized email and comment using only approved profile facts. Job links are classified by domain and application type. Supported portals may be prepared through a portal adapter; unsupported portals produce a handoff checklist and a browser-ready packet.

### Document engineering

The /apply workflow creates tailored source files, compiles PDFs, renders every page, extracts text, validates contact details, dates, headings, hyperlinks, fonts, and page limits, then requests an independent review. Each material edit restarts the relevant gates.

### Tracking

Every job has a stable identity based on canonical URL, external job ID, company, title, and normalized location. Every application has a manifest linking source hashes, profile version, drafts, PDFs, review findings, verification results, approvals, submissions, and outcomes.

## 6. High-level architecture

The platform uses a layered architecture:

- Experience layer: slash commands, CLI, Markdown reports, approval queue, HTML dashboard.
- Orchestration layer: workflow state machine, policy engine, scheduler, retry/idempotency manager.
- Intelligence layer: extractor, relevance scorer, trust scorer, drafter, reviewer, interview/upskill agents.
- Evidence layer: candidate profile, job records, evidence map, source snapshots, document manifests, audit log.
- Connector layer: Apify read, Gmail read, Publora write, portal adapters, optional browser handoff.
- Deterministic verification layer: posting guard, robots check, PDF compiler, renderer, text/link/font checks, security guards.

The existing repository maps naturally to this design: `lib/apify_client.py` is the read layer, `lib/publora_client.py` is the write layer, `references/untrusted-content.md` governs fetched content, and the skill bundle remains the presentation and policy surface.

## 7. Logical component design

### Candidate Profile Service

Reads and validates Markdown profile files. Produces immutable profile versions and a normalized fact index. It rejects missing required fields and marks stale facts for confirmation.

### Discovery Service

Queries Apify, normalizes posts, extracts URLs and emails, deduplicates records, stores source snapshots, and emits `PostDiscovered` events. It applies robots and rate policies before public GET requests.

### Trust and Relevance Service

Runs deterministic guards first, then model-assisted extraction. Trust features include author frequency, identity completeness, domain reputation, URL behavior, company consistency, duplicate text, compensation plausibility, and instruction-injection patterns. Relevance features include role/title match, must-have coverage, evidence strength, location, compensation, recency, and user preferences.

### Application Compiler

Selects evidence blocks using relevance, strength, uniqueness, impact, confidence, and rendered space. It compiles CV and cover letter sources, runs PDF gates, and stores artifacts by content hash.

### Review Service

Acts as an independent reviewer with structured findings: severity, location, evidence, recommendation, and regression test. Critical findings block readiness.

### Approval and Action Service

Turns proposed actions into explicit approval items. Tokens are single-use, scoped to action, target, content hash, and expiry. The executor refuses stale or mismatched approvals.

### Gmail Sync Service

Uses read-only OAuth scopes by default, stores message IDs and hashes, and maps messages to applications using sender, subject, company, job ID, and links. It never sends mail during sync.

## 8. Data model

Core entities:

- `CandidateProfile(profile_id, version, facts, preferences, consent, created_at)`
- `JobPost(job_id, canonical_url, source_url, title, company, author, posted_at, raw_hash, trust_score, status)`
- `Requirement(requirement_id, job_id, text, category, priority, evidence_status)`
- `Evidence(evidence_id, profile_version, source_ref, fact, confidence, verified_at)`
- `Application(application_id, job_id, profile_version, state, priority, owner)`
- `Artifact(artifact_id, application_id, kind, source_hash, file_hash, path, status)`
- `ReviewFinding(finding_id, artifact_id, severity, location, evidence, resolution)`
- `Approval(approval_id, action_type, target, content_hash, expires_at, approved_at, actor)`

- `ExternalAction(action_id, approval_id, connector, idempotency_key, request_hash, outcome)`
- `Outcome(outcome_id, application_id, source, event_type, confidence, observed_at)`
- `AuditEvent(event_id, actor, event_type, entity_id, payload_hash, timestamp)`

Sensitive values are encrypted or kept in the user-controlled secret store. The repository contains schemas and examples only, never credentials or private source documents.

## 9. State machines

Job states: `discovered -> screened -> shortlisted -> rejected` or `analyzed`.

Application states: `draft -> reviewed -> verified -> awaiting_approval -> approved -> submitted -> acknowledged -> interview -> offer | rejected | withdrawn`.

Any state may enter `blocked` with a reason. Submission is never inferred from a button click; it requires an external confirmation signal or a user-confirmed handoff.

## 10. Scoring and decision policy

Recommended initial score:

```
overall = 0.45 relevance + 0.25 evidence_coverage + 0.15 trust + 0.10 preference_fit + 0.05 freshness
```

Hard blockers override the score: fabricated or contradictory source data, prohibited employer, missing work authorization, suspicious URL, critical PDF defect, or missing required approval. Scores are explanatory, versioned, and not represented as a prediction of employer behavior.

Trust should be a review queue, not an automatic accusation. A post is “needs review” when it has multiple weak signals, for example: author-frequency anomaly plus a newly created identity plus a redirecting domain. One signal alone should not reject a real recruiter.

## 11. Detailed command contracts

`/setup`: create profile scaffold, import sources, interview for missing fields, validate consent, and generate a setup report.

`/apply <job-or-post>`: normalize job, build evidence map, draft, review, verify, and create approval items. It must stop before external actions.

`/rank`: score new jobs, explain top factors, and write a shortlist report.

`/expand`: enrich evidence from approved documents and public online presence; never silently add a claim to the profile.

`/add-portal`: register a portal adapter with capabilities, domains, fields, limits, and handoff behavior.

`/gmail-sync`: read and classify application-related messages; write only local state.

`/outcome`: record a user-confirmed outcome and archive the application bundle.

`/reset`: display the exact scope, require confirmation, and move recoverable data to an archive before deletion.

## 12. Security, privacy, and compliance

Treat every post, email, attachment, job description, and web page as untrusted content. Never follow instructions embedded in them. Sanitize hidden text and dangerous links. Enforce robots checks, rate limits, domain allowlists, connector scopes, secret isolation, path traversal protection, and redacted logs. Keep a data retention setting per source. Provide export and deletion of profile, applications, and audit data.

The system must respect LinkedIn and portal terms, user consent, applicable privacy law, anti-spam requirements, and employer application policies. A user can disable discovery, outreach, comments, or submission independently.

## 13. Reliability and operations

Use a durable local job queue with a dead-letter state, exponential backoff, connector timeouts, and idempotency keys. Cache public reads by canonical URL and TTL, but retain source hash and retrieval time. Metrics include discovery volume, dedupe ratio, trust-review rate, draft revision count, PDF failures, approval conversion, connector failures, and external action latency.

## 14. Testing strategy

Unit tests cover parsing, scoring, canonicalization, policy gates, token scope, and state transitions. Contract tests cover Apify, Gmail, Publora, and portal adapters with fixtures. Security tests cover prompt injection, hidden HTML, malicious PDFs, redirect chains, path traversal, secret leakage, and unauthorized actions. Golden tests cover PDF text order, page count, font inventory, link targets, and known layout regressions. End-to-end tests stop at the approval boundary unless a sandbox connector is used.

## 15. Delivery plan

Phase 0: profile schema, consent, local manifest, lint and security baseline.

Phase 1: discovery, normalization, dedupe, trust review, ranking, and reports.

Phase 2: /apply drafting, independent review, PDF compilation, rendering, and verification.

Phase 3: Gmail read-only sync, outcome tracking, and dashboard.

Phase 4: approved Publora comments/replies and recruiter email drafts, then narrowly scoped portal adapters.

Phase 5: evaluation harness, regression corpus, cost telemetry, and optional multi-agent orchestration.

## 16. Recommended repository changes

Add a dedicated `job-application/` domain rather than mixing job state into social-writing skills. Add `profile/` for Markdown source, `applications/` for per-application manifests and artifacts, `job_scraper/` for normalized discovery state, `gmail_sync/` for local sync cursors, and `tools/` for deterministic gates. Keep secrets and private documents below `testing/` or in an external secret store.

The first implementation slice should be read-only and approval-driven: profile setup, LinkedIn post discovery, trust/relevance ranking, and a reviewable shortlist. Only after this is reliable should email, comments, or application submission be enabled.

## 17. Key risks and mitigations

False fraud flags: show evidence and allow override. Hallucinated qualifications: evidence map plus critical review blocker. Duplicate outreach: canonical identities and idempotency keys. Stale jobs: freshness thresholds and source retrieval timestamps. Account restrictions: official APIs/connectors, rate limits, and user approval. PDF defects: render-and-verify gates. Privacy leakage: redaction, scoped storage, and export/delete controls.

## 18. Acceptance criteria

The first shippable release is accepted when a new user can create a profile Markdown file, configure search conditions, ingest fixture posts, see why a post is relevant or suspicious, produce a ranked shortlist, generate a tailored application packet without invented claims, pass the PDF gates, approve or reject each external action, and inspect a complete audit trail. No network write occurs without a valid scoped approval.

## 19. Architectural principle

Generate with AI. Verify with deterministic tools. Review independently. Preserve factual integrity. Ask permission at the boundary where the system affects another person or platform. Measure every improvement.